

Exercícios de Estruturas de Dados - 1

Programação Orientada a Dados

Prof. Me. Otávio Parraga



1. Intercalação de Listas:

- Crie uma função `intercalar_listas(lista1, lista2)` que receba duas listas e retorne uma nova lista onde os elementos de `lista1` e `lista2` sejam intercalados. Caso uma lista tenha mais elementos que a outra, os elementos restantes devem ser adicionados ao final.

```
lista1 = [1, 3, 5]
lista2 = [2, 4, 6, 8, 10]
print(intercalar_listas(lista1, lista2))
# Saída esperada: [1, 2, 3, 4, 5, 6, 8, 10]
```

2. Tuplas de Pares Ordenados:

- Crie uma função `criar_pares(lista1, lista2)` que receba duas listas e retorne uma lista de tuplas onde cada tupla contenha um elemento de `lista1` e o respectivo elemento de `lista2` na mesma posição. Caso uma lista seja maior, os elementos extras devem ser ignorados.

```
lista1 = ['a', 'b', 'c']
lista2 = [1, 2, 3, 4]
print(criar_pares(lista1, lista2))
# Saída esperada: [('a', 1), ('b', 2), ('c', 3)]
```

3. Remoção de Duplicatas:

- Implemente uma função `remover_duplicatas(lista)` que receba uma lista e retorne uma nova lista contendo os mesmos elementos, mas sem duplicatas. A ordem dos elementos deve ser mantida.

```
lista = [1, 2, 2, 3, 4, 4, 5]
print(remover_duplicatas(lista))
# Saída esperada: [1, 2, 3, 4, 5]
```

4. Filtrando Tuplas por Critério:

- Crie uma função `filtrar_tuplas(lista_tuplas, limite)` que receba uma lista de tuplas onde cada tupla contém um nome e uma idade, e retorne apenas as tuplas onde a idade seja maior que um determinado limite.

```
lista_tuplas = [('Ana', 23), ('João', 19), ('Maria', 30), ('Pedro', 17)]
print(filtrar_tuplas(lista_tuplas, 20))
# Saída esperada: [('Ana', 23), ('Maria', 30)]
```

5. Soma de Elementos Pares:

- Escreva uma função `soma_pares(lista)` que receba uma lista de números inteiros e retorne a soma dos números pares.

```
lista = [1, 2, 3, 4, 5, 6]
print(soma_pares(lista))
# Saída esperada: 12
```

6. Inverter Tupla:

- Crie uma função `inverter_tupla(tupla)` que receba uma tupla e retorne uma nova tupla com os elementos na ordem inversa.

```
tupla = (1, 2, 3, 4, 5)
print(inverter_tupla(tupla))
# Saída esperada: (5, 4, 3, 2, 1)
```

7. Produto Cartesiano de Listas:

- Escreva uma função `produto_cartesiano(lista1, lista2)` que retorne o produto cartesiano de duas listas, ou seja, uma lista de tuplas com todas as combinações possíveis de elementos de `lista1` com `lista2`.

```
lista1 = [1, 2]
lista2 = ['a', 'b']
print(produto_cartesiano(lista1, lista2))
# Saída esperada: [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
```

8. Concatenar Listas de Tuplas:

- Crie uma função `concatenar_listas_tuplas(lista1, lista2)` que recebe duas listas de tuplas e retorna uma nova lista onde as tuplas em posições correspondentes são concatenadas (os valores dentro das tuplas são unidos em uma nova tupla).

```
lista1 = [(1, 2), (3, 4)]
lista2 = [(5, 6), (7, 8)]
print(concatenar_listas_tuplas(lista1, lista2))
# Saída esperada: [(1, 2, 5, 6), (3, 4, 7, 8)]
```